

```
#include <stdio.h>
#include <ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char item) {
    if (top >= MAX - 1) {
        printf("Stack Overflow\n");
    } else {
        stack[++top] = item;
    }
}

char pop() {
    if (top == -1) {
        return -1;
    } else {
        return stack[top--];
    }
}

int precedence(char symbol) {
    if (symbol == '^') return 3;
    else if (symbol == '*' || symbol == '/') return 2;
    else if (symbol == '+' || symbol == '-') return 1;
    else return 0;
}

void infixToPostfix(char infix[]) {
    int i = 0, j = 0;
    char postfix[MAX], item, x;

    while ((item = infix[i++]) != '\0') {
        if (isalnum(item)) {
            postfix[j++] = item;
        } else if (item == '(') {
            push(item);
        } else if (item == ')') {
            while ((x = pop()) != '(') {
                postfix[j++] = x;
            }
        } else {
            while (top != -1 && precedence(stack[top]) >= precedence(item)) {
                postfix[j++] = pop();
            }
            push(item);
        }
    }

    while (top != -1) {
        postfix[j++] = pop();
    }
    postfix[j] = '\0';

    printf("Postfix Expression: %s\n", postfix);
}
```

```
$ gcc infix_to_postfix.c -o infix_to_postfix
$ ./infix_to_postfix
```

Enter Infix Expression: (A+B)*(C-D)

Postfix Expression: AB+CD-*

Execution Trace / Conversion Steps:

Symbol Read	Stack Content	Postfix Expression
-------------	---------------	--------------------

'('	(	
'A'	(	A
'+'	(+	A
'B'	(+	AB
')'		AB+
'*'	*	AB+
'('	*(	AB+
'C'	*(	AB+C
'-'	*(-	AB+C
'D'	*(-	AB+CD
')'	*	AB+CD-
End of String	Empty	AB+CD-*

Final Output | AB+CD-*